

**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES**

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Code of Conduct:

I Oviya M / 192321117 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE:** CSA37

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Question 1 (Bloom's Level 3)

A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Answer:

Introduction

Testing is an engineering activity: it is planned, designed, executed, measured and improved just like any other engineering task. Ad-hoc testing in a banking application, where testers click around without a plan, leaves requirements uncovered and lets defects such as wrong balances or duplicate debits reach production. The approach must therefore be redesigned around a defined process.

Redesigned Approach

- **Test planning:** define scope, risks (money movement, security), entry/exit criteria, schedule and roles.
- **Systematic test design:** derive test cases from requirements using black-box techniques (equivalence partitioning, boundary value analysis, decision tables, state transition) and white-box techniques (statement and branch coverage).
- **Tools:** a test management tool (TestRail/Jira Xray) for traceability, Selenium/Appium for automation, JMeter for load, and a defect tracker (Jira/Bugzilla).
- **Standards:** adopt IEEE 829 for test documentation, ISO/IEC 29119 for the test process and a defect-reporting template with severity and priority.

How Quality and Defect Detection Improve

Every requirement maps to at least one test case (traceability), so no feature is left untested. Techniques such as boundary analysis target the places where defects cluster. Automated regression suites catch breakages after each build, and metrics (defect density, test coverage, defect removal efficiency) show whether quality is improving.

Techniques to Replace Ad-hoc Testing

Requirement-based test design, risk-based testing, boundary value analysis, decision-table testing, checklists, reviews and inspections, and automated regression testing.

Practical Example

For a fund-transfer feature with a daily limit of Rs. 50,000, boundary value analysis gives tests at 0, 1, 49,999, 50,000 and 50,001. Ad-hoc testers typically try only 1,000 or 5,000 and miss the off-by-one defect at the limit. A decision table covering account status (active/frozen), balance (sufficient/insufficient) and limit (within/exceeded) yields eight systematic cases that expose a defect where frozen accounts could still send money. Such a defect would be missed without structure.

Conclusion

Treating testing as an engineering activity with planning, design techniques, tools and standards gives repeatable, measurable and effective testing, which reduces production failures in the banking application.

Question 2 (Bloom's Level 3)

An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Answer:

Structured Testing Across SDLC

- **Requirements:** review requirements for clarity and testability and prepare the acceptance test plan. Example: 'Withdrawal allowed only in multiples of 100, maximum Rs. 20,000 per transaction.'
- **Design:** review design documents, prepare the system and integration test plan and design test cases.
- **Coding:** unit testing and code reviews by developers (statement and branch coverage for the PIN-check and balance-check logic).
- **Integration:** test the interfaces between the card reader, PIN module, bank server and cash dispenser.
- **System testing:** execute all functional and non-functional cases (security, performance, reliability) on the complete ATM.
- **Acceptance/deployment:** user acceptance testing, then regression testing for each patch.
- **Maintenance:** retest and regression test after every change.

Application to ATM Withdrawal

Test cases include: valid card and PIN with sufficient balance (cash dispensed, balance reduced); wrong PIN three times (card blocked); amount greater than balance (declined); amount not a multiple of 100 (rejected); amount above the limit; dispenser out of cash; network failure after debit (transaction must reverse); and card removed during the transaction. Each case has an ID, input, expected result and traceability to a requirement.

How Process Consistency Reduces Defects

A defined process means every release is tested the same way, with the same entry and exit criteria, so results are repeatable and not dependent on one tester's memory. Early review finds requirement defects when they are cheapest to fix, and the defect log feeds back into improved test cases.

Expected Outcomes

Consistent results across ATMs, fewer withdrawal failures, earlier defect detection, reduced rework cost, measurable quality (coverage and defect trends) and higher customer trust.

Question 3 (Bloom's Level 3)

A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Answer:

Definitions

- **Error (mistake):** a human action that produces an incorrect result, e.g. a developer writes *if (balance > amount)* when it should be $\geq$.
- **Defect (fault/bug):** the flaw in the code or document that results from the error, i.e. the wrong operator in the source.
- **Failure:** the observable deviation when the defective code runs, e.g. a user with exactly Rs. 500 cannot transfer Rs. 500.

Relationship

Error leads to Defect, and Defect (when executed under the right conditions) leads to Failure. A defect may remain hidden until a particular input triggers it. In the login scenario, a developer error (forgetting case-insensitive email comparison) creates a defect, and the failure is that 'User@mail.com' is rejected although registered. In the transaction scenario, a rounding error in the interest calculation causes the failure of a wrong displayed amount.

Effect of Improper Understanding

If a failure is reported as 'login is broken', developers cannot locate the defect. Confusing defects with failures leads to fixing symptoms instead of root causes, and testers may close a bug because it did not reproduce even though the defect still exists. Misclassification also distorts metrics such as defect density.

Improving Defect Reporting Clarity

- Use a unique ID, short title, environment (browser, OS, build number) and module.
- List exact steps to reproduce, with test data.
- State the expected result and the actual result separately.
- Attach screenshots, logs and the severity and priority.
- Example: 'Login fails for email with uppercase letters. Steps: 1) open login page 2) enter User@mail.com and valid password 3) click Login. Expected: dashboard opens. Actual: Invalid credentials message. Severity: High.'

Conclusion

Precise use of the terms error, defect and failure makes root-cause analysis faster and bug reports actionable, so the mismatches are resolved with less rework.

Question 4 (Bloom's Level 3)

A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Answer:

Principles Applied

- **Defect clustering:** a small number of modules contain most defects (the Pareto 80/20 pattern). Test effort should be focused there.
- **Testing shows presence of defects:** testing cannot prove absence, so risky modules need repeated attention.
- **Exhaustive testing is impossible:** we must prioritize using risk.
- **Early testing:** start testing in requirements and design to prevent clusters.
- **Pesticide paradox:** the same tests stop finding bugs, so test cases must be reviewed and renewed.
- **Absence-of-errors fallacy:** bug-free software that does not meet user needs is still a failure.

Guiding Test Planning

Use the defect repository to rank modules by defect density. Allocate more time, senior testers and deeper techniques (code reviews, branch coverage) to the top modules, and

lighter testing to stable modules. Re-assess after every release because clusters move.

Prioritizing Test Cases

Score each test case by (a) business impact, (b) probability of failure (past defect count, code complexity, recent change) and (c) usage frequency. Execute high-risk and high-impact cases first (smoke and critical-path), then medium, then low. Remove duplicates and add new cases for areas where the pesticide paradox appears.

Efficiency Gains

More defects are found per testing hour, critical defects are found earlier, schedule slippage is reduced and effort is not wasted on low-risk modules.

Real-time Example

In an online shopping project, 70% of defects were found in the Payment and Cart modules. The team applied extra boundary, integration and security tests there, ran the full payment regression suite daily and tested the product catalogue only at sprint end. Defect leakage to production fell and release delays stopped.

Question 5 (Bloom's Level 3)

A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Answer:

Origin of Defects

Defects originate at every stage: unclear or missing requirements, wrong design decisions, coding mistakes, inadequate testing, wrong build or configuration, and poor change control. Human error, time pressure and miscommunication are the root causes.

Defect Classification

- **Requirements/specification defects:** missing, ambiguous or conflicting requirements.
- **Design defects:** wrong algorithm, interface or data-structure design.
- **Coding defects:** control/logic flow, data handling, computation, interface and initialization errors.
- **Testing defects:** wrong test case, wrong test data or environment.
- Also by severity (critical to low), priority, phase of origin and module.

Using Repository Data

Analyse the repository with Pareto and trend charts to find which module, defect class and phase produce most defects. Example: if 40% of repeated defects are 'boundary condition' defects in the billing module, add boundary test cases there and add a coding checklist item

for boundaries. Recurring defects also become regression test cases so they cannot silently return.

Improvements to Avoid Recurrence

- Perform root cause analysis (5-Why, fishbone) for every major defect.
- Record fields consistently: class, origin phase, root cause, fix, module.
- Review repository metrics at each release retrospective.
- Update checklists, coding standards and training based on the findings.
- Link each defect to the test case that found it and to a regression test.

Expected Quality Improvements

Reduced repeat defects, lower defect density, better test coverage in weak areas, faster triage, improved defect removal efficiency and a measurable drop in post-release defects.

Question 6 (Bloom's Level 3)

A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Answer:

Identifying the Root Cause

- **Reproduce** the crash with exact steps, device, OS version, network type and build.
- **Collect evidence:** crash logs, stack trace, server logs and transaction IDs.
- **Narrow down** by varying inputs: amounts, payee types, network (Wi-Fi/4G/offline), device memory, OS version.
- **Apply principles:** defect clustering (the transfer module is the cluster), early testing and exhaustive testing being impossible (so risk-based selection of the combinations most likely to fail).
- Typical causes: unhandled null payee data, timeout without retry handling, memory leak, API contract mismatch, concurrency when the user taps 'Pay' twice.

How Systematic Testing Reduces Such Issues

Planned tests cover functional paths (valid and invalid amounts, limits), negative cases (insufficient balance, wrong OTP), interruption tests (call, low battery, app switched to background), network failure tests, compatibility tests across devices and performance and stress tests. Each is derived from requirements and risks, not from guesswork.

Examples

- Boundary tests at the per-transaction limit expose a crash at exactly the limit amount.
- Switching from Wi-Fi to mobile data during a transfer exposes an unhandled timeout exception.

- Rapid double-tap on Pay exposes a duplicate request and a null-pointer error.

Improvements in Practice

Add automated regression tests (Appium) for the transfer flow, run tests on a device cloud, add crash analytics (Firebase Crashlytics), perform code reviews on exception handling, run load and stress tests before release, use staged rollout and keep every crash as a permanent regression case.

Question 7 (Bloom's Level 3)

A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Answer:

Differences

- **Testing** is the process of executing software to detect failures and evaluate quality. It is done by testers (and developers for unit tests), it can be planned and automated, and its output is defect reports.
- **Debugging** is the process of locating the defect that caused a failure, finding its cause and fixing it. It is done by developers, it is investigative, and its output is corrected code.
- Testing shows *that* a defect exists; debugging finds *where and why*. Testing is scheduled and measurable; debugging is not predictable.

Roles in Software Quality

Testing provides evidence about quality and risk, while debugging removes the defect. Quality needs both: testing without debugging leaves known defects in place, and debugging without testing does not verify that the fix works or that nothing else broke.

Effect of Improper Usage

If developers 'test' by just debugging what they happen to notice, only obvious defects are found and there is no coverage. If testers try to debug, they spend time on code fixes instead of finding more defects. Skipping retesting after a fix lets regressions through. Example: a developer fixes a login crash and ships it without a re-test; the fix breaks password reset.

Structured Integrated Approach

- Tester executes planned tests and logs a defect with reproduction steps.
- Developer reproduces, debugs (breakpoints, logs, root-cause analysis) and fixes the defect.
- Tester performs confirmation (re-)testing of the fix.

- Regression tests verify that nothing else is broken.
- Defect is closed and recorded in the repository for prevention analysis.

Question 8 (Bloom's Level 3)

An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Answer:

Origin of Defects

Browsers use different rendering engines (Blink, Gecko, WebKit) and implement HTML, CSS and JavaScript standards differently. Defects arise from non-standard or unsupported CSS properties, JavaScript API differences, outdated browser versions, different handling of cookies, fonts and form validation, and screen-size differences.

Cross-browser Testing Strategy

- **Analyse usage data** to pick the browser/OS/device matrix (e.g. Chrome, Edge, Firefox, Safari on Windows, macOS, Android, iOS).
- **Prioritize** the matrix by risk and traffic (exhaustive testing is impossible).
- **Design tests** for critical flows: search, product page, cart, checkout and payment, login.
- **Test types:** functional, UI/layout, responsive, performance and accessibility on each combination.
- **Execute early and regularly** in the CI pipeline, with automated tests plus manual exploratory checks on key browsers.
- **Log defects** with browser name and version and compare with a baseline browser.

How Systematic Testing Improves Reliability

A defined matrix and repeatable scripts guarantee that every release is checked on each supported browser, so layout breaks or a non-working 'Place Order' button in Safari are caught before users see them. Results are comparable between releases.

Tools and Techniques

Selenium WebDriver or Playwright for automation, BrowserStack or LambdaTest for real devices and browsers, Chrome DevTools for responsive checks, Percy/Applitools for visual comparison, caniuse.com for compatibility checks, and techniques such as equivalence partitioning of browsers/devices and pairwise (combinatorial) testing to reduce the number of combinations.

Question 9 (Bloom's Level 3)

A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Answer:

Categorization of the Defects

- **By origin:** coding defects (logic errors) arising in the implementation stage.
- **By type:** computation/algorithm defects (wrong formula for overtime, tax slab or PF), control-flow defects (wrong condition such as using > instead of >=), data defects (wrong rounding or data type), and boundary defects (tax slab limits).
- **By severity:** critical, since wrong pay affects employees and legal compliance.
- **Possible requirement defects:** tax rules misinterpreted at the specification stage.

How Understanding Defect Classes Improves Testing

Each class suggests the right technique. Computation defects need test cases with known expected salaries (oracle) and boundary value analysis at tax slabs; control-flow defects need branch and decision coverage; data defects need tests on rounding and large values. Knowing the dominant class helps allocate tests where defects are probable.

Prevention in Future Releases

- Code reviews and walkthroughs of salary formulas by a second developer.
- Unit tests for each formula, e.g. basic 30,000 + overtime 5 hours should equal a hand-computed expected value.
- Static analysis tools to detect suspicious conditions.
- Update coding checklists with 'rounding, boundaries, slab limits'.
- Record defects in the repository and add each as a regression test.

Improvements in Testing Strategy

Create a regression suite with parallel comparison against the previous version (back-to-back testing), use decision tables for combinations of allowances and deductions, test month-end and leave-without-pay cases, and add UAT with finance staff validating real payslips.

Question 10 (Bloom's Level 3)

A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Answer:

Redesigned Process

- Analyse requirements and identify test conditions.
- Choose test design techniques and design test cases (ID, objective, preconditions, input, expected result, postconditions).
- Review the test cases.
- Prepare data and environment, execute, compare actual with expected results and log defects.
- Measure coverage and refine.

Importance of Structured Test Case Design

Structured test cases give coverage of requirements, are reproducible and reusable for regression, can be reviewed, allow effort estimation and measurement, and serve as evidence of quality. Random tests repeat the same obvious inputs, miss rarely used conditions and cannot be repeated exactly.

Effect on Defect Detection

Example: a field accepts ages 18 to 60. Random testers enter 25 and 30 and find nothing. Equivalence partitioning and boundary value analysis produce 17, 18, 19, 59, 60, 61, which expose a defect where 60 is rejected (the code uses < 60). Fewer test cases find more defects.

Suitable Techniques

Equivalence partitioning, boundary value analysis, decision-table testing, state-transition testing, use-case testing, pairwise testing, error guessing and exploratory testing as a complement, and white-box techniques (statement, branch, condition coverage). Reviews of test cases and traceability matrices further improve effectiveness.

Question 11 (Bloom's Level 3)

A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Answer:

Analysis Using Testing Principles

Testing shows presence of defects, and defect clustering suggests starting with the modules that exchange data (registration, billing, pharmacy, lab). Early testing of interfaces and data models, and risk-based prioritization (patient safety first) are applied. Compare the same patient record in each module to locate the point of divergence.

Origin of Defects

Inconsistent data models or field formats between modules, duplicated data stores, missing validation, incorrect data mapping or transformation in interfaces, unsynchronized updates (race conditions), incomplete requirements on data ownership, and untested integration after changes.

Systematic Testing for Data Integrity

- **Unit and validation tests:** field formats, mandatory fields and ranges.
- **Integration tests:** data created in registration must appear identically in billing and pharmacy.
- **Database tests:** referential integrity, constraints, no orphan records.
- **CRUD and end-to-end tests:** update a patient's allergy in one module and verify it everywhere.
- **Concurrency and rollback tests:** simultaneous updates and failed transactions leave no partial data.
- **Security and privacy tests** as per healthcare rules.

Improvements in Defect Tracking and Reporting

Use a defect tool with fields for module, data entity, severity (patient safety = critical) and root cause, attach data samples (with identifiers masked), link defects to requirements and test cases, track status through a defined life cycle and review integrity-related trends in every release.

Question 12 (Bloom's Level 3)

A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Answer:

Analysis Using a Defect Repository

A defect repository is a central database of all defects with attributes such as ID, module, class, severity, phase found, root cause and fix. Repeated defects show that the data is either not analysed or the lessons are not fed into test design. Query the repository for duplicates, reopened defects and defects with the same root cause across releases.

How Repository Data Supports Test Design

Frequent defect classes become test conditions, e.g. repeated date-handling defects lead to specific date boundary test cases. Defect-prone modules receive more tests, and each fixed defect is converted into a regression test. The data also guides checklists for reviews.

Identifying Patterns

- Pareto chart: which 20% of modules give 80% of defects.

- Trend chart: defects per release and per phase.
- Classification count: which defect type recurs (e.g. validation).
- Reopen rate: fixes that fail retests indicate weak root-cause fixing.
- Example: 15 of 40 defects are input-validation defects in the registration screens, so the team adds validation test cases and developer training.

Improvements

Hold a defect review at the end of every release, do root-cause analysis, record preventive actions, improve coding standards and checklists, automate regression for fixed defects, and track defect removal efficiency and escaped defects as release metrics.

Question 13 (Bloom's Level 3)

A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Answer:

Identifying Gaps

The principle that exhaustive testing is impossible implies risk-based planning, and the failure shows that performance and load risk was not considered. Gaps: no load or stress tests at expected peak (for example exam registration time), no concurrency tests, no scalability or database connection limit tests, and missing non-functional requirements such as 'support 5,000 simultaneous users with response under 3 seconds'.

Impact of Test Planning and Coverage

A test plan that lists non-functional risks, workload models, environments and exit criteria ensures that performance is tested at all. Coverage should include not only functional coverage but also coverage of load levels, user journeys (login, video streaming, quiz submission) and data volumes. Low coverage of these scenarios leaves hidden failure modes.

Better Test Strategy and Reliability

- Load testing at expected peak, stress testing beyond it to find the breaking point, spike testing for sudden surges and soak (endurance) testing for memory leaks.
- Monitor CPU, memory, response times and error rates during tests.
- Fix bottlenecks (slow queries, missing caching, thread limits) and retest.
- The result is predictable behaviour at peak load.

Enhancements

Define performance requirements early, use JMeter, Gatling or Locust with production-like data and environments, include performance tests in the CI/CD pipeline, use auto-scaling

and CDN, monitor production and review incidents to update the test suite.

Question 14 (Bloom's Level 3)

A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Answer:

Role of the Tester in SDLC

- **Requirements:** review for ambiguity and testability and start acceptance test design.
- **Design:** review architecture and design and plan system and integration tests.
- **Development:** support unit and integration testing, review code and prepare test data.
- **Testing and release:** execute tests, report defects and give quality status.
- **Maintenance:** regression testing and defect analysis.

Impact of Early Involvement

Early testing finds defects when they are cheapest to correct, since the cost of a defect rises sharply in later phases. Testers bring a 'what can go wrong' view that improves requirements, and the test design itself acts as a review that exposes missing details.

Practical Example

In a hospital appointment app, the requirement said 'users can reschedule an appointment'. The tester asked what happens within 2 hours of the slot and when the doctor is unavailable. The requirement was clarified before coding. Without this, the defect would have been found in system testing and required design and code changes.

Changes in Collaboration Practices

- Include testers in requirement, design and sprint-planning meetings.
- Adopt shift-left practices and the 'three amigos' (business, developer, tester) discussion for each story.
- Use shared definitions of done and acceptance criteria.
- Pair testers and developers; share test cases for unit tests.
- Hold joint defect triage and retrospectives.

Question 15 (Bloom's Level 3)

A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Answer:

Identifying Gaps

Using the principles of exhaustive testing being impossible and defect clustering, measure what has been covered: requirement coverage (traceability matrix), code coverage (statement, branch, condition) and input coverage. Untested requirements, uncovered branches (such as error-handling paths) and missing partitions of input data are the gaps.

How Proper Coverage Ensures Reliability

Coverage shows how much of the system has been exercised. Higher and well-targeted coverage reduces the chance that untested paths hide defects, and gives objective evidence for release decisions. Coverage is only a measure; it does not prove correctness, so test quality still matters.

Improving Test Case Design

Example: a discount function gives 10% for amount over 1,000 and 20% over 5,000. Tests with 500 and 2,000 pass. Equivalence partitions are $\leq 1,000$, 1,001-5,000 and $> 5,000$, and boundaries are 1,000, 1,001, 5,000 and 5,001. Adding the boundary cases exposes the defect at exactly 5,000. Also add negative and invalid inputs.

Techniques for Better Validation

Equivalence partitioning, boundary value analysis, decision tables, state-transition testing, branch and condition coverage, mutation testing to check test quality, requirement traceability matrix, reviews of test cases and exploratory testing to find unexpected failures.

Question 16 (Bloom's Level 3)

A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Answer:

Improving Defect Reporting

Report each defect as soon as it is found, in a standard template in a defect tracking tool, so developers can reproduce it quickly without back-and-forth questions.

Importance of Clear Definitions

When the team agrees on terms (error, defect, failure, severity, priority, status), each defect is classified the same way, filtering and counting work properly and triage is faster. Example: severity is the technical impact, priority is the urgency of fixing. A parcel-tracking page that shows a wrong delivery status for all parcels is High severity and High priority; a spelling mistake is Low severity and Low priority.

Structured Reports Enhance Communication

A good report contains ID, title, module, environment and build, steps to reproduce, expected and actual results, severity, priority, attachments (screenshots, logs), reporter, assignee and status. Example: 'Shipment status not updating after delivery scan. Steps: 1) scan parcel P1001 as delivered 2) open tracking page. Expected: Delivered. Actual: In transit.' The developer can reproduce it instantly and the manager can see impact.

Improvements in Reporting Standards

- Adopt a standard template (IEEE 829 incident report) and mandatory fields.
- Define a defect life cycle (New, Assigned, Fixed, Retest, Closed, Reopened).
- Set service levels for triage by severity.
- Review reports for clarity and remove duplicates.
- Publish dashboards of open defects, ageing and trends.

Question 17 (Bloom's Level 3)

A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Answer:

Process-based Approach

Define a single organization-wide test process: planning, test design, execution, defect management, reporting and closure, with templates, entry and exit criteria and roles. Refer to ISO/IEC 29119 and process-improvement models such as TMMi, which assign a maturity level to the process.

How Standardized Processes Improve Quality

Every module is tested by the same method, so results are comparable and gaps are visible. New team members become productive faster, work is repeatable, measurement is possible and improvement is based on data.

Example

Three modules (orders, inventory, billing) each had their own habits: one used test cases, one used checklists and one tested ad-hoc. After standardization, each module follows the same template: test plan, test cases derived with boundary and equivalence techniques, peer review, execution with a common defect tool, and an exit criterion of 100% critical tests passed and no open high-severity defects. Defect leakage across modules becomes uniform and low.

Methods to Ensure Consistency

- Common templates and a test management tool.
- Peer reviews and audits against the process.

- Common metrics: coverage, defect density, defect removal efficiency.
- Training and a documented test policy.
- Regular process retrospectives and continuous improvement.

Question 18 (Bloom's Level 3)

A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Answer:

Why Edge Cases Were Missed

Testers focused on typical inputs, time pressure led to limited testing (exhaustive testing is impossible), requirements did not state limits, and test design relied on intuition instead of techniques. The pesticide paradox also applies: the same tests were repeated and no new cases were added.

How Boundary Testing Helps

Many defects occur at the edges of input ranges because of off-by-one mistakes in comparison operators (< versus <=). Boundary value analysis tests the minimum, just above minimum, just below maximum and maximum, plus values just outside the range. Few tests, high defect detection.

Including Edge Cases

- Identify ranges, limits, sizes and lengths in the requirements.
- Example: password length 8 to 12 characters. Tests: 7, 8, 9, 11, 12, 13.
- Include empty or null input, zero, negative values, maximum file size, leap day (29 Feb), time-zone and date boundaries, special characters and very large data.
- Add each production edge-case failure to the regression suite.

Improvements in Test Design

Use boundary value analysis with equivalence partitioning, decision tables, error guessing from the defect repository, exploratory testing sessions on edge conditions, a requirements review that makes limits explicit, and test case reviews with checklists of common edge cases.

Question 19 (Bloom's Level 3)

A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Answer:

Addressing the Problem

Set up a central defect repository (Jira, Bugzilla, Redmine) where every defect is entered with standard fields and tracked through its life cycle, replacing e-mails, chats and spreadsheets.

How Documentation Improves Defect Management

Documented defects can be reproduced, assigned, prioritized and verified. Nothing is lost or forgotten, there is a history of fixes for reference, and responsibility is clear. Management gets accurate status and metrics.

Structured Repository Supports Analysis

Because every defect has fields for module, class, severity, phase of origin and root cause, the repository can produce Pareto charts of defect-prone modules, trend charts per release, defect ageing, defect density and defect removal efficiency. For example, analysis may show 50% of defects are requirement defects, so the company invests in requirement reviews.

Improvements in Documentation Practices

- Mandatory template: ID, title, description, steps to reproduce, expected and actual result, environment, severity, priority, attachments, status and owner.
- Defined life cycle and status rules.
- Link defects with test cases and requirements.
- Record root cause and fix details when closing.
- Regular review of quality of defect records and training of staff.

Question 20 (Bloom's Level 3)

A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

Answer:

Redesigned Testing Approach

- Apply early testing: review requirements for transaction rules (limits, charges, atomicity).

- Apply risk-based testing: transactions involving money are highest risk.
- Plan: scope, environment with production-like data, entry and exit criteria.
- Design: derive cases systematically from requirements, with traceability.
- Execute: manual and automated, with defect logging and regression after every fix.
- Measure and improve using metrics from the defect repository.

How Structured Testing Improves Quality

It ensures that all transaction types (transfer, deposit, withdrawal, bill pay), valid and invalid inputs, failures mid-transaction and concurrent use are covered. Defects are found earlier and are repeatable, and changes are protected by regression tests, so repeated failures stop.

Examples

- Atomicity: simulate failure after debit but before credit; the transaction must roll back.
- Concurrency: two simultaneous withdrawals from the same account must not overdraw the balance.
- Boundary: transfer of exactly the available balance and balance plus 1 paisa.
- Idempotency: network retry must not post the same transaction twice.

Techniques to Enhance Defect Detection

Boundary value analysis, equivalence partitioning, decision tables and state-transition testing for the transaction life cycle (initiated, authorised, posted, reversed), database and reconciliation testing, security and performance testing, automated regression, code reviews, and analysis of the defect repository to direct effort at defect-prone areas.